OPERATING SYSTEMS

ASSIGNMENT 3 – CO3

Name	K. Vimala
Register Number	1924260099
Course Code	CSA04

Rubric-based solutions for Questions 1–7

OPERATING SYSTEMS

ASSIGNMENT 3 – CO3

Rubric-Based Solutions | K5 – Evaluate

Student detail	To be filled by student
Name	_____
Register Number	_____
Course	Operating Systems (CSA04)
Course Outcome	CO3 – Memory Management

Coverage: Linking and loading, paging, segmentation, virtual memory, demand paging, page replacement and disk management.

Important submission note: This document provides written solutions, calculations and code for the CO3 questions (Q1–Q7). The assignment also requires each student to execute programs, collect measurements/screenshots and submit a GitHub repository. Do not report unmeasured experimental values as if they were collected.

Q1 – Linking, Loading and Address Binding (6 Marks)

Problem: A bank distributes a large executable to all branches whenever its encryption library is updated. The aim is to reduce update size and simplify maintenance.

1. Compile-time, load-time and execution-time address binding

Binding type	Explanation	Evaluation
Compile-time	If the starting memory location is known, the compiler generates absolute addresses. If the location changes, the program must be recomp	Simple, but inflexible.
Load-time	The compiler generates relocatable code. The loader selects the memory location and adjusts addresses when loading.	More flexible than compile-time binding.
Execution-time	Address translation occurs while the program runs, usually with hardware support from the MMU.	Most flexible; supports process relocation and virtual memory.

2. Roles of linker and loader

• The linker combines object files, resolves external symbols and links required libraries to create an executable.

• The loader creates the process image, maps the executable and required shared libraries into memory, performs required relocations and transfers control to the program.

3. Static linking versus dynamic linking

Factor	Static linking	Dynamic/shared linking
Executable size	Usually larger because library code is copied into the executable.	Usually smaller because library code remains in shared library files.
Memory use	Each process may contain its own library copy.	Read-only library pages can be shared by processes.
Start-up	May avoid locating/loading separate shared libraries; actual start-up depends on program and system.	Dynamic loader must locate and map dependencies.
Updating	Library fix generally requires rebuilding and redistributing executable.	Compatible shared library can be updated without rebuilding each application.
Failure	Executable is less dependent on external library files.	Missing/incompatible shared library can prevent launch.

4. Position-independent code (PIC)

Position-independent code can execute correctly when mapped at different virtual addresses. Shared libraries are commonly built as PIC, allowing the same read-only code pages to be mapped into multiple processes without modifying the code for each process.

5. Required implementation commands

```
# Static build (example)
gcc -static main.c -o bank_static

# Shared library and dynamically linked executable
gcc -fPIC -shared crypto.c -o libcrypto_demo.so
gcc main.c -L. -lcrypto_demo -Wl,-rpath,'$ORIGIN' -o bank_dynamic

# Inspect dependencies and executable sizes
ls -lh bank_static bank_dynamic
ldd bank_dynamic

# Observe memory mappings for a running process
cat /proc/<PID>/maps
```

For run-time loading, include and use dlopen("./libcrypto_demo.so", RTLD_NOW), then obtain a function address with dlsym(handle, "encrypt_data"). Check the returned handle and symbol pointer for errors, and call dlclose(handle) when finished.

6. Experimental investigation and evaluation

• Measure start-up time with repeated runs (for example, /usr/bin/time -f '%e seconds' ./bank_static) and compare the median; avoid relying on one run.

• Measure memory with /usr/bin/time -v or /proc//status. Run two processes and inspect shared mappings in /proc//smaps to identify shared pages.

• Replace the shared library with a compatible version and run the program again. Record the library version and observed behaviour.

• Temporarily test with the library unavailable in a controlled test directory. Record the dynamic-loader error; restore the library after the test.

Conclusion: Dynamic linking is the better default for the bank because a compatible encryption-library security update can be deployed once and shared by branch applications. It reduces executable duplication and can share library pages. The bank must manage library versions, compatibility, secure library paths and rollback. Static linking may still be suitable for isolated tools that must run without external library dependencies.

Alternative concept: load the library only when needed

Use dlopen() immediately before the first encryption operation and dlsym() to find the required function. When encryption is not used, the library need not be mapped into the process, which can reduce its initial memory footprint and may reduce start-up work. The first encryption request has extra loading and symbol-resolution delay. If the library is missing, dlopen() returns NULL; the program must check dlerror(), report a clear failure and either disable the feature safely or stop the operation. Static/dynamic start-up measurements and memory measurements must be collected on the student's Linux system.

Complexity: Library lookup is generally bounded by loader/linker data structures and is not usefully expressed as a per-input algorithmic Big-O. The key comparison is operational overhead: eager dynamic loading occurs at process start; dlopen() defers that cost until the feature is used.

Q2 – Paging (8 Marks)

Problem: Contiguous allocation leaves external fragmentation on a multi-user server. Paging allows a process's pages to occupy non-contiguous physical frames.

1. External fragmentation

External fragmentation occurs when free memory is split into many small non-adjacent holes. The total free memory may be sufficient, but no single hole is large enough for a requested contiguous allocation.

```
Example physical memory:
| Process A | free 2 MB | Process B | free 3 MB | Process C | free 1 MB |
Total free = 6 MB, but a 4 MB contiguous request cannot fit.
```

2. Logical-to-physical address translation

For page size 1,024 bytes: page number p = logical address / 1,024 (integer division); offset d = logical address % 1,024. If the page-table entry is a valid frame f, physical address = f × 1,024 + d. If p is outside the page table, the address is invalid; if the entry is –1, the page is not resident and causes a page fault.

Logical	Page	Offset	Frame	Result
0	0	0	5	5×1024+0 = 5120
1500	1	476	2	2×1024+476 = 2524
3100	3	28	7	7×1024+28 = 7196
6200	6	56	3	3×1024+56 = 3128
9000	8	808	—	Invalid address (page 8 outside 0–7)
2049	2	1	–1	Page fault

3. Page-table size (4 KB pages)

4 KB = 2^{12} bytes. Assuming one 4-byte page-table entry per virtual page:

• 32-bit address space: $2^{32} / 2^{12} = 2^{2}$■ = 1,048,576 entries. Size = 1,048,576 × 4 = 4,194,304 bytes = 4 MiB.

• 48-bit address space: 2■■ / $2^{12} = 2^{3}$■ entries. Size = 2^{3}■ × 4 = 2^{3}■ bytes = 256 GiB for a single-level page table.

The 48-bit result shows why practical systems use multi-level, sparse or other page-table structures rather than allocating one enormous flat table for every process.

4. Effective access time (TLB)

Let memory access time be m and TLB lookup time be t. With a TLB hit ratio h and one memory access on a hit, a common simplified model is EAT = h(t + m) + (1 – h)(t + 2m). A hit requires TLB lookup plus one memory access; a miss requires TLB lookup plus page-table access and data access. Example assumption: t = 10 ns and m = 100 ns.

Hit ratio	Calculation	EAT
80%	0.8(110) + 0.2(210)	130 ns
95%	0.95(110) + 0.05(210)	115 ns
99%	0.99(110) + 0.01(210)	111 ns

Without a TLB, a single-level page-table lookup followed by the data access takes approximately 2m = 200 ns in this simplified model. Actual systems may use caches and multi-level tables.

5. Why paging removes external fragmentation but keeps internal fragmentation

• A process is divided into fixed-size pages and physical memory into same-size frames. Any free frame can hold any page, so a large contiguous physical region is unnecessary.

• Internal fragmentation remains because the last page of a process may not be completely filled. With page size S, waste in the final page is between 0 and S–1 bytes.

6. Page-size evaluation

Page size	Advantages	Disadvantages
1 KB	Less waste in the last page; finer-grained allocation.	More pages and larger page tables; more TLB pressure.
4 KB	Practical balance; widely used default on many systems.	Moderate page-table and internal-fragmentation cost.
2 MB	Fewer page-table entries; can improve TLB reach and large-region locality.	Potentially higher internal waste; allocation and fragmentation constraints.

7. Page-table designs, protection and sharing

• Hierarchical page table: divides the page table into levels; allocate lower-level tables only for used address ranges. Translation may require walking multiple levels on a TLB miss.

• Hashed page table: hashes the virtual page number to find a mapping; useful for large address spaces.

• Inverted page table: one entry per physical frame, recording which process and virtual page occupy it. Its table size is related to physical memory, not every process's full virtual address space.

• Protection is provided with page-table permission bits such as read, write, execute and user/supervisor. Shared libraries and shared-memory regions can map the same physical frame into multiple processes with suitable permissions.

8. Complete the code – TODO 1

```
#include <stdio.h>
#define PAGE_SIZE 1024
#define PAGES 8
int page_table[PAGES] = {5, 2, -1, 7, 0, -1, 3, -1};

int main(void) {
    unsigned logical[] = {0, 1500, 3100, 6200, 9000, 2049};
    for (int i = 0; i < 6; i++) {
        unsigned a = logical[i];
        unsigned p = a / PAGE_SIZE;
        unsigned d = a % PAGE_SIZE;
        printf("%5u -> page %u offset %6u : ", a, p, d);
        if (p >= PAGES) printf("invalid address\n");
        else if (page_table[p] == -1) printf("page fault\n");
        else printf("physical %d\n", page_table[p] * PAGE_SIZE + (int)d);
    }
    return 0;
}
```

Expected output:

```
    0 -> page 0 offset      0 : physical 5120
 1500 -> page 1 offset    476 : physical 2524
 3100 -> page 3 offset     28 : physical 7196
 6200 -> page 6 offset     56 : physical 3128
 9000 -> page 8 offset    808 : invalid address
 2049 -> page 2 offset      1 : page fault
```

9. Change PAGE_SIZE to 2048

With PAGE_SIZE = 2048 and the same page-table entries: 0 → p0,d0, physical 10240; 1500 → p0,d1500, physical 11740; 3100 → p1,d1052, physical 5148; 6200 → p3,d56, physical 14392; 9000 → p4,d808, physical 808; 2049 → p1,d1, physical 4097. No address is outside the 8-entry table and none maps to −1.

10. Experimental checks and alternative: inverted page table

```
getconf PAGESIZE
/usr/bin/time -v ./paging_program
# Test first/last byte of a page and an address beyond the process's mapped range.
```

For 100 randomly sized processes, calculate waste as (ceil(size/page_size) × page_size) – size for each process and sum the results. Repeat for 1 KB, 4 KB and 2 MB using the same generated process sizes. Record the random seed so the comparison is reproducible.

Alternative: An inverted page table stores one entry per physical frame, so space is proportional to physical memory. It can save substantial table space compared with a full per-process virtual page table. Translation typically hashes (process ID, virtual page number) and searches a bucket/chain; collisions can add lookup work. A multi-level table uses

a bounded level walk on a TLB miss but allocates tables for used virtual regions. The best choice depends on address-space sparsity, physical-memory size and lookup performance.

Conclusion: A 4 KB page is a reasonable general-purpose choice for the college server because it balances internal fragmentation, page-table size and TLB efficiency. Large pages can be enabled selectively for large, stable workloads.

Q3 – Segmentation (6 Marks)

1. Segment-table address translation

A logical address is (segment number s, offset d). Each segment-table entry contains a base address and a limit. If d is greater than or equal to the limit, the address is invalid and a protection fault occurs. Otherwise, physical address = base + d.

Segment	Base	Limit	Example offset	Result
Code (0)	1000	400	120	1120 valid
Data (1)	5000	800	799	5799 valid
Stack (2)	9000	600	650	Invalid: offset exceeds limit

2. Segmentation with paging

In segmented paging, the segment number selects a segment-table entry. That entry identifies the page table for the segment. The offset is split into a page number and page offset; the page table maps the page to a physical frame. The physical address is frame number × page size + page offset. Segment limits and page permissions are checked during translation.

3. Programmer's view, protection and sharing

• Segmentation matches logical program units such as code, data, heap and stack, which can have different sizes and growth patterns.

• Each segment can have separate permissions. Code can be read/execute but not writable; data can be read/write; stack can be read/write.

• A common code segment can be shared by several processes, while each process keeps private data and stack segments.

4. Fragmentation and scheme comparison

Scheme	Fragmentation	Translation / tables	Sharing and protection
Pure paging	Internal; no external fragmentation.	Page table maps fixed-size pages to frames.	Page-level permissions and shared frames.
Pure segmentation	External fragmentation; variable-size segments.	Segment table with base and limit; simple translation after checks.	Natural protection and sharing by logical segment.
Segmented paging	Small internal waste within pages; avoids external fragmentation.	Segment table points to page tables; more complex translation.	Segment-level organisation plus page-level mapping and protec

5. Experimental investigation

• Create a small program with a segment table and translate eight (segment, offset) pairs, including offsets exactly at the limit and one beyond the limit.

• A segmentation fault occurs when a process accesses memory outside a valid mapped region or violates permissions. The CPU raises a fault; the kernel delivers SIGSEGV and normally terminates the process unless handled.

• For 20 randomly sized segments, simulate allocation and deallocation, then measure total free space and the largest free hole. External fragmentation can be reported as 1 – (largest free hole / total free space), when total free space is non-zero.

• Compare table storage: pure segmentation needs one entry per segment; paging needs entries for mapped virtual pages; segmented paging needs segment descriptors plus page-table entries.

6. Alternative: pure paging

Pure paging divides all memory into fixed-size pages. It removes external fragmentation, but can waste up to one page minus one byte in the final page of each allocation. Page tables may be larger because code, data and stack are represented as pages rather than a few logical segments. Translation uses a page-table lookup (often cached by a TLB). Code can still be shared by mapping the same read-only physical frames into multiple processes; permissions are attached to pages. Compared with segmented paging, pure paging is simpler and is well suited to modern hardware, but

it provides less direct representation of logical program regions.

Conclusion: Modern general-purpose operating systems rely mainly on paging because fixed-size frames simplify physical-memory allocation and eliminate external fragmentation. Protection and sharing can be implemented with page permissions and shared mappings.

Q4 – Virtual Memory and Demand Paging (8 Marks)

1. Page-fault handling steps

• The CPU references a page whose page-table entry indicates it is not currently resident; hardware traps to the operating system.

• The OS checks whether the reference is valid and permitted. If invalid, it reports an access violation; if valid, it locates the page on storage or creates a zero-filled page.

• The OS obtains a free frame. If none is available, it selects a victim page and writes it back if modified.

• The required page is read into the selected frame; the page table and TLB are updated.

• The interrupted instruction is restarted and execution continues.

2. Effective access time calculations

Given memory access = 100 ns and page-fault service time = 8 ms = 8,000,000 ns. Approximate EAT = (1 – p) × 100 + p × 8,000,000 ns, where p is the page-fault probability. This simplified calculation treats the fault-service time as including the major fault-handling cost.

Fault rate p	Calculation	EAT
1/1,000 = 0.001	0.999×100 + 0.001×8,000,000	8,099.9 ns ≈ 8.10 µs
1/100,000 = 0.00001	0.99999×100 + 0.00001×8,000,000	179.999 ns ≈ 180 ns
1/1,000,000 = 0.000001	0.999999×100 + 0.000001×8,000,000	107.9999 ns ≈ 108 ns

3. How 24 GB can run with 16 GB RAM

Demand paging loads a page into RAM only when it is referenced. The program's virtual address space can therefore be larger than physical RAM, provided its actively used working set fits sufficiently well and backing storage is available. Inactive pages can remain on disk. If the active working set is larger than available RAM, repeated page faults and swapping can cause thrashing and severe slowdown.

4. Maximum page-fault rate for 10% slowdown

A 10% slowdown permits EAT = 110 ns. Solve 110 = (1 – p)100 + p(8,000,000). Thus p = 10 / 7,999,900 ≈ 0.00000125 = 1.25 × 10■■, or about one fault per 800,000 memory references. This is an approximate upper bound under the stated simplified model.

5. Copy-on-write, working set and thrashing

• With copy-on-write, fork() initially lets parent and child map the same physical pages as read-only. When one process writes, the OS creates a private copy of the affected page. This avoids copying all memory immediately.

• The working set is the set of pages a process has referenced during a recent time window. Allocating enough frames for the working set reduces repeated faults.

• Thrashing occurs when processes spend most of their time paging rather than executing. The OS can detect high page-fault rates and low CPU utilisation, reduce the degree of multiprogramming, suspend processes or allocate more frames.

6. Frame allocation

Equal allocation gives each process the same number of frames. It is simple but ignores process size. Proportional allocation assigns frames according to process size: frames_i = (size_i / total size) × available frames. Priority and working-set information can improve this decision. The OS must reserve frames for the kernel and important services.

7. Experimental investigation

```
# Linux tools for observation
/usr/bin/time -v ./memory_test
# Compare sequential and random page-touch patterns.
# Record Minor page faults, Major page faults and elapsed time.
# For fork/COW, record memory before fork and after child writes.
```

Build a controlled test that allocates and touches memory gradually. Compare sequential access with random access and record minor/major faults and elapsed time. To study copy-on-write, record process memory before fork(), then let the child write to selected pages and record memory again. Plot page faults against frame count for the chosen reference string. Use a VM or a safely configured memory limit; do not exhaust a personal or shared system's memory.

Conclusion: Improving locality is the first recommendation because it reduces page faults without hardware cost. If the radiologist's active working set genuinely exceeds available RAM, adding RAM is important; relying on disk-backed paging for sustained high-performance image processing will be slow.

Alternative: 32 GB RAM

With 32 GB RAM, the 24 GB workload may fit in physical memory, so the steady-state page-fault rate can be much lower after pages are loaded. Under the simplified model, if p = 0, EAT is approximately 100 ns; if paging continues at a measured non-zero rate, use the same EAT formula with that measured p. Compare the price of 16 GB additional RAM with the measured time saved and the operational cost of slow processing. Do not claim p = 0 without measuring.

Q5 – Page-Replacement Techniques (8 Marks)

1. Algorithms

• FIFO removes the page that entered memory earliest. It is simple, but may remove a frequently used page and can suffer Belady's anomaly.

• LRU removes the page that has not been used for the longest time. It uses recent history and performs well when locality is strong.

• Optimal removes the page whose next use is farthest in the future. It gives the minimum possible faults for a known reference string, but future references are not known in a real system.

• Clock/second-chance maintains a reference bit. A page with bit 1 is given a second chance (bit reset to 0); a page with bit 0 is replaced. It approximates LRU with low overhead.

2. Given reference string and expected results

Reference string: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

Frames	FIFO faults	LRU faults	Optimal faults
3	15	12	9
4	10	8	8

For the Belady reference string 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5, FIFO gives 9 faults with 3 frames and 10 faults with 4 frames. This is Belady's anomaly: adding a frame increases FIFO page faults.

3. Complete the code – LRU TODO

```
int lru(const int ref[], int len, int frames)
{
    int f[MAXF], last[MAXF], used = 0, faults = 0;
    for (int i = 0; i < len; i++) {
        int k = find(f, used, ref[i]);
        if (k >= 0) { last[k] = i; continue; }
        faults++;
        if (used < frames) {
            f[used] = ref[i];
            last[used] = i;
            used++;
        } else {
            int victim = 0;
            for (int j = 1; j < frames; j++)
                if (last[j] < last[victim]) victim = j;
            f[victim] = ref[i];
            last[victim] = i;
        }
    }
    return faults;
}
```

When this function is inserted into the supplied program, the expected output is:

```
3 frames: FIFO 15, LRU 12, Optimal 9
4 frames: FIFO 10, LRU 8, Optimal 8
Belady string, FIFO: 3 frames 9 faults, 4 frames 10 faults
```

4. Why FIFO anomaly occurs; why LRU does not

FIFO's replacement order depends on arrival time, not on the set of pages currently resident. With the supplied string, the 4-frame FIFO run makes 10 faults, compared with 9 for 3 frames. LRU has the stack property: the pages held with n frames form a subset of those held with n+1 frames at each point in the reference sequence. Therefore, increasing frames cannot increase LRU's fault count.

5. Evaluation for a web server

Algorithm	Strength	Limitation
FIFO	Very simple; low bookkeeping.	Ignores usage; anomaly possible.

Algorithm	Strength	Limitation
LRU	Good when recent use predicts near-future use.	Exact implementation needs timestamps/list updates or hardware support.
Optimal	Best theoretical fault count; useful as a benchmark.	Cannot know future references.
Clock	Efficient approximation; one reference bit per frame.	May not match exact LRU fault count.

6. Complexity analysis

For a simple implementation that linearly searches up to F frames for every reference, FIFO and LRU take O(NF) time for N references and F frames; space is O(F). The supplied Optimal implementation searches the remaining reference string for each candidate victim, giving $O(N^2F)$ worst-case time and O(F) frame storage (excluding the input string). A hash-based lookup can improve page-hit search, while exact LRU still needs recency maintenance.

7. Experimental investigation and recommendation

Generate one reference string with repeated accesses to a small hot set (strong locality) and another with uniformly random page numbers. For each algorithm, test 3–10 frames, count faults and measure runtime for 100,000 references. Plot faults against frame count and verify one small case by hand. Use the same strings for all algorithms to make the comparison fair.

Recommendation: LRU or a clock/second-chance approximation is a reasonable choice for the web server because workloads often have locality. Exact LRU gives strong recency behaviour but has higher bookkeeping overhead. Clock is practical because it uses a reference bit and a circular scan. Choose between them using the measured fault count and runtime from the actual workload.

Alternative – Clock instead of exact LRU: Each frame needs a reference bit (plus the page number and normal frame metadata). On access, set the bit. On replacement, inspect frames circularly: clear and skip a 1; replace the first frame found with 0. This avoids updating a full timestamp or recency list on every reference. A scan can take O(F) in the worst case for one replacement, while the amortised work is usually modest. Compare clock's measured faults with LRU on the same test strings.

Q6 – Disk Management (4 Marks)

1. Disk preparation and booting

• Low-level formatting creates sectors and sector headers on the physical disk (normally performed by the manufacturer).

• Partitioning divides the disk into logical regions using a partition table such as GPT or MBR.

• Logical formatting creates a file system (for example ext4 or NTFS) within a partition, including metadata structures.

• The boot process begins with firmware (BIOS/UEFI), which selects a boot device and starts boot code or an EFI boot manager. The bootloader loads the OS kernel and initial RAM filesystem; the kernel initialises hardware and mounts the root file system.

2. Bad blocks

A disk may detect defective sectors using error-correcting codes and internal remapping to spare sectors. The operating system or file system can also avoid known bad blocks and report I/O errors. Regular monitoring and backups are required; RAID is not a substitute for backup.

3. RAID comparison for six 4 TB disks

RAID	Usable capacity (6 × 4 TB)	Reliability / speed
RAID 0	24 TB	Striping; high performance, but any disk failure can lose the array.
RAID 1	4 TB if all six mirror one another; 12 TB if configured as three mirrored pairs	Mirroring; can survive failures depending on which disks fail; read performance can improve.
RAID 5	(6–1)×4 = 20 TB	Single distributed parity; survives one disk failure; write penalty due to parity updates.
RAID 10	(6/2)×4 = 12 TB	Striped mirrors; good performance and redundancy; can survive multiple failures if they are not in the s

RAID 1 capacity depends on layout. The 4 TB figure is a six-way mirror; 12 TB is the common three-pair mirrored layout. Actual formatted capacity is lower than the raw TB calculation.

4. Swap space

Swap provides backing store for inactive memory pages and can support hibernation where configured. Size depends on RAM, workload, peak memory pressure and hibernation requirements. For a hospital records server, place swap on reliable storage with sufficient free capacity and monitoring. A dedicated swap partition/file can be used; a separate physical disk may reduce contention only if it does not become a performance bottleneck. Avoid placing the only copy of critical records in swap.

5. Linux inspection commands

```
lsblk
sudo fdisk -l
swapon --show
free -h
df -h
```

6. Probability of data loss

The assignment does not provide a numerical disk failure rate or time interval, so a numeric probability cannot be calculated without inventing an input. If each disk has independent failure probability q during the chosen interval: a two-disk mirror loses data if both members fail, approximately q^2 (more precisely, depends on repair timing and assumptions). RAID 5 loses data if two or more disks fail before rebuild; for six disks, assuming identical independent q, P(loss) = 1 – (1–q)^6 – 6q(1–q)^5. Insert the measured/assumed q and state the interval.

Conclusion: For a hospital records server, RAID 10 is a strong performance and availability choice when capacity budget permits; RAID 6 is also attractive where surviving two disk failures is the priority. Maintain independent, tested backups and off-site copies.

Alternative – RAID 6

RAID 6 stores two parity blocks per stripe and can survive any two disk failures. For six 4 TB disks, usable capacity = (6–2)×4 TB = 16 TB. Writes have extra parity calculation and update work compared with RAID 5, so small random

writes can be slower. The benefit is protection against a second disk failure during rebuild. For critical records, RAID 6 may be preferred over RAID 5 when the extra fault tolerance is more important than the additional capacity and write overhead.

Q7 – Design a Memory-Management Plan from First Principles (10 Marks)

Scenario: Edge device has 2 GB RAM, no swap disk, a real-time control task that must not be delayed by page faults, an ML task requiring 1.5 GB and a logging task. The design must leave room for future tasks.

Stage 1 – Interpret the problem (2 Marks)

Task	Memory / timing need	Page-fault tolerance
Real-time control	Small, bounded working set; strict worst-case response deadline.	Must not incur page faults during its critical execution.
Machine-learning	Up to 1.5 GB; compute and data buffers may be large.	Can tolerate some delay, but frequent faults harm throughput.
Logging	Small buffers; mostly sequential writes.	Can tolerate buffering delays; should not block control task.

Measurable requirements: (1) the control task's critical code and data remain resident and locked; (2) its worst-case loop delay stays below the control deadline; (3) ML and logging memory use remains within a reserved budget; (4) the system does not rely on swap; (5) reserve memory for the kernel and future services.

Stage 2 – Compare candidate mechanisms (3 Marks)

Mechanism	Benefits	Limitations / use
Contiguous allocation	Simple, predictable placement for a fixed real-time region.	Difficult to grow; external fragmentation. Suitable only if fixed region can be reserved.
Paging	Non-contiguous physical frames; protection and sharing; avoids external fragmentation.	Page faults can cause unacceptable delays. Use with resident/locked critical pages.
Segmentation	Separates code, data, stack and permissions.	External fragmentation; less common as the main physical-memory mechanism.
Demand paging	Loads pages only when first referenced; efficient for large, sparse data.	Page faults introduce latency; unsuitable for the critical control loop.

Design decision: Use Linux paging for general process isolation, but pre-fault and lock the real-time task's critical memory using mlock()/mlockall() (subject to permissions and resource limits). Reserve its memory before starting the ML task. Run ML with a controlled memory budget and allow it to use the remaining available RAM. Keep logging buffered and rate-limited so it cannot consume the control task's reserved memory or CPU time.

Stage 3 – Implement and verify (3 Marks)

```
/* Illustrative Linux C outline: compile with gcc -O2 -Wall -Wextra rt_lock.c */
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <sys/mman.h>
#include <time.h>

int main(void) {
    const size_t bytes = 1024 * 1024;
    volatile unsigned char *buf = malloc(bytes);
    if (!buf) { perror("malloc"); return 1; }

    /* Touch every page before entering the critical loop. */
    for (size_t i = 0; i < bytes; i += 4096) buf[i] = 0;

    if (mlock((void *)buf, bytes) != 0) {
        perror("mlock");
        free((void *)buf);
        return 1;
    }

    /* Demonstration only: use a measured, bounded control loop here. */
    for (int k = 0; k < 100000; k++)
        buf[(size_t)k % bytes]++;

    munlock((void *)buf, bytes);
    free((void *)buf);
    return 0;
}
```

Run the same loop with and without mlock(), collect many latency samples using a monotonic clock, and report median, 99th percentile and maximum observed delay. These are observed maxima, not a formal proof of worst-case execution time. Check /proc//status (VmLck) and system limits. Ensure the locked memory is within the device's budget.

For the simulator, represent the control, ML and logging tasks as page-reference sequences. Give the control task a fixed resident set and simulate ML/logging page replacement only in their assigned frames. Verify that the control task's pages are never selected for eviction. Report the frame allocation and fault counts.

Stage 4 – Defend the design (2 Marks)

The design protects the control task by reserving and locking its working set before real-time operation. ML uses the remaining memory under a cap, while logging uses bounded buffers. If a fourth 500 MB task is added, the 2 GB device may not have enough capacity after kernel, control, ML and logging reservations. The system must reject/defer the new task, reduce the ML working set, compress/stream its data, or upgrade RAM; it must not silently steal the control task's locked memory. A remaining limitation is that mlock() prevents paging of selected memory but does not by itself guarantee CPU scheduling deadlines, interrupt latency or device-I/O timing.

Conclusion: Use a hybrid design: Linux paging for flexible workloads, locked and pre-touched memory for the real-time control task, bounded memory for ML and logging, and no dependence on swap. Validate the timing with repeatable measurements under peak load.

Alternative – Real-time OS without virtual memory

A real-time OS with no virtual memory can provide more predictable address translation and avoid demand-page faults. The control task can have a statically allocated region and more deterministic timing. However, the ML task's available memory is limited by the physical 2 GB and by the memory reserved for the kernel, control and logging; a 1.5 GB ML requirement leaves little room for growth. Adding new tasks requires careful static memory planning and may require rebuilding/reconfiguring the system. Linux with locked memory offers more flexible process isolation and general-purpose services, but needs careful control of page faults, scheduling and memory limits.

Rubric checklist: Stage 1 requirements and timing constraints; Stage 2 mechanism comparison and justified choice; Stage 3 executable program, measurements and simulator evidence; Stage 4 defence, fourth-task analysis and limitation. Include your actual output screenshots and measurement tables in the submitted report.

Appendix – GitHub and Evidence Submission Checklist

• Create repository: CSA04--Assignment3.

• Create folders Task01 through Task07. Put source code, scripts, inputs, outputs and screenshots in the matching folder.

• Add README.md with student name, register number, Linux distribution/version, gcc version, exact compile commands and run instructions.

• Commit progressively: at least one meaningful commit per task. Do not upload everything in one final commit.

• Include measured start-up time, memory use, page faults, algorithm fault counts, plots and command outputs where required.

• Use the supplied expected outputs as a correctness check. Your own system measurements may differ; report the environment and method.

• Add the working GitHub URL to the report cover page and submit it as required by the faculty.

Important: The assignment explicitly assesses design, implementation, analysis and evaluation. A written explanation alone is not complete evidence for tasks that require execution and measurement.